

講義 6：讀 SAP Table——航班模型與 SELECT

對應練習：ex06 | 答案程式：ZR_TR06_SAP_TABLE

本講重點

- 資料字典 (DDIC) 與透明表：SE11 看定義、SE16N 看資料
- **Global Type**：型別從哪裡來 (內建型別 / Local Type / Global Type)，為什麼業務欄位要引用 DDIC 型別
- SAP 練習用航班資料模型：SCARR / SPFLI / SFLIGHT
- `SELECT ... INTO TABLE`、`SELECT SINGLE`、`WHERE`、`UP TO n ROWS`、`ORDER BY`
- `sy-subrc` 與 `sy-dbcnt`
- 為什麼不用 `SELECT *`、為什麼避免 `SELECT ... ENDSELECT`

1. 資料字典 (Data Dictionary) 與透明表 (Transparent Table)

SAP 的資料表定義集中在**資料字典 (Data Dictionary，DDIC)**，用 SE11 檢視：欄位、型別 (Data Element)、鍵、外鍵關係都在這裡。透明表 (transparent table) 就是資料庫裡真實存在的表。

交易代碼	用途
SE11	看表的 定義 (欄位、型別、鍵)
SE16N	看表的 資料 (開發/測試機查數據)

宣告變數時直接參考 DDIC 型別，是實務最標準的寫法：

```
DATA gv_carrid TYPE scarr-carrid.           " 跟表欄位同型別
DATA gs_carrier TYPE scarr.                  " 整列結構
DATA gt_carriers TYPE STANDARD TABLE OF scarr. " 內表：一列 = 一筆 SCARR
```

好處：表定義改了，程式的變數自動一致；而且帶著欄位的語意 (長度、轉換規則、檢核表)。這個「引用型別而非寫死」的觀念正式名稱是 **Global Type**，下一小節先把它跟前面學過的型別放在一起比較。

1.1 內建型別、Local Type、Global Type：型別從哪裡來

宣告變數時，型別有三種來源，從「最靠近程式」到「最靠近系統」：

層級	寫法	定義在哪	特性
內建型別	<code>DATA a TYPE c LENGTH 3.</code>	ABAP 語言本身	長度、語意都寫死在這一行
Local Type	<code>TYPES ty_x ... 再 DATA a TYPE ty_x.</code> (講義 3、4)	這支程式裡	只有本程式看得到，別的程式要用得再抄一份
Global Type	<code>DATA a TYPE scarr-carrid.</code> 、 <code>TYPE scarr</code> 、 <code>TYPE s_carr_id</code>	資料字典 (SE11)	全系統共用 同一份 定義，帶著標籤、F4 說明、檢核表

Global Type 涵蓋的範圍：**Data Element**（如 `s_carr_id`）、**表格欄位**（如 `scarr-carrid`，背後其實就是該欄位使用的 Data Element）、**Structure / 整列**（如 `scarr`）、**Table Type**（表格型別，講義 25 詳述）。

為什麼要引用，不要寫死：SAP 標準的物料號碼欄位 `MATNR`，舊版是 18 碼，S/4HANA 加長成 40 碼。

```
DATA lv_matnr TYPE mara-matnr.    " Global Type：升級後自動變 40 碼，程式一行不用改
DATA lv_matnr TYPE c LENGTH 18.   " 寫死長度：升級後資料被默默截斷成 18 碼，也不會報錯
```

寫死長度的欄位不會當機、不會噴訊息，只是資料悄悄壞掉。把型別的定義權交給系統唯一的來源（資料字典），程式只負責「引用」，這就是 Global Type 存在的意義。

判斷準則：

- 純區域暫存（迴圈計數器 `TYPE i`、只在本段用的旗標）→ 內建型別就好
- 代表業務資料（航空公司、物料、票價、日期.....）→ 一律引用 Global Type：`TYPE` 表-欄位
- 只有本程式用的欄位組合 → Local Type（`TYPES`）；多支程式或 **Function Module** 都要用 → 升級成 DDIC 的 Structure / Table Type（講義 25）

實務上，程式裡代表業務資料的變數絕大多數都是引用 Global Type，直接寫 `TYPE c LENGTH n` 來放業務欄位反而是少見、也不建議的寫法。講義 25 會進一步教怎麼動手建自己的 Domain / Data Element / Table Type，以及「該重用標準型別還是自建」的判斷。

2. 航班資料模型（訓練標準教材）

SAP 內建一組練習用資料表，本課程到期末都用它：

表	內容	鍵欄位
SCARR	航空公司主檔	CARRID
SPFLI	航線（起訖機場、時刻）	CARRID, CONNID
SFLIGHT	航班（日期、票價、座位）	CARRID, CONNID, FLDATE

三張表用 CARRID（+CONNID）串起來：一家公司有多條航線，一條航線有多個日期的航班。沒有資料時先執行報表 `SAPBC_DATA_GENERATOR`（SE38 跑一次）產生測試資料。

另外每張 SAP 表第一欄幾乎都是 `MANDT`（client）：Open SQL 會自動只撈當前 client 的資料，WHERE 不用（也不要）自己寫 `MANDT`。

2.1 延伸參考：常用 SAP 表速查

航班模型是練習用的。實務上接到 MM / SD / FI 的需求，第一步通常是「這個資料存在哪張表」。查法：

- 已知表名：SE11 看定義、SE16N 看資料。
- 不知道表名：SE84（Repository Information System）用表的說明文字搜尋（講義 25）；或在交易畫面的欄位上按 F1 → **Technical Information**，可以看到這個欄位屬於哪張表或哪個結構。

各模組最常用的表（本系統 S/4HANA 1909）：

模組	表	內容
組織 / 共用	T001	公司代碼
	T001W、T001L	廠別、儲存地點
	TCURR、TCURX	匯率、幣別小數位 (講義 21 §2.2)
	T006	計量單位
物料 (MM)	MARA、MAKT	物料主檔一般資料、物料說明 (依語言)
	MARC、MARD、MBEW	物料的廠別資料、儲存地點資料、評價資料
	EKKO、EKPO	採購單表頭、明細
	EBAN	請購單
	MATDOC	物料憑證 (入庫、發料、調撥)，S/4HANA 新表
銷售 (SD)	KNA1、KNVV	客戶一般資料、客戶銷售資料
	VBAK、VBAP、VBEP	銷售單表頭、明細、交貨排程
	LIKP、LIPS	交貨單表頭、明細
	VBRK、VBRP	發票表頭、明細
	VBFA	單據流程 (銷售單 → 交貨單 → 發票的前後關係)
	PRCD_ELEMENTS	單據的定價條件，S/4HANA 新表
財務 (FI)	BKPF、BSEG	會計憑證表頭、明細
	ACDOCA	總帳行項目 (Universal Journal)，S/4HANA 新表
	SKA1、SKB1	會計科目 (科目表層、公司代碼層)
	LFA1、LFB1	供應商一般資料、公司代碼資料
業務夥伴	BUT000	Business Partner 一般資料，S/4HANA 建客戶 / 供應商的入口

講師另外提供一批 R/3 時代 (2000 ~ 2008 年) 整理的 SAP 表清單文件，可以當模組入門參考；但那些清單沒有上表粗體的 S/4HANA 新表，裡面列的部分舊表在 S/4HANA 也已經改變，以下面這段為準。

2.2 S/4HANA 的資料表變化：新表、合併、Proxy

S/4HANA 跑在 HANA 記憶體資料庫上，SAP 趁機把 R/3 時代很多「為了效能而存的重複資料」拿掉：合計、索引、狀態這類表不再另外存，改成需要時從明細即時算出來。對寫程式的人來說，舊表會落入下面幾種情況：

情況	意思	程式該怎麼做
A. 舊表名變成 CDS View	實體表已經沒了，舊名稱是一個從新表即時計算的 CDS View	讀取還能用；新程式改讀新表

情況	意思	程式該怎麼做
B. Proxy (替代物件)	舊表還在，但讀它時系統自動改讀一個 CDS View，資料是從新表即時算出來的	讀取還能用； 不要寫入 ；新程式改讀新表
C. 表還在但不寫資料	舊表不再存資料，欄位或資料搬到其他表	讀舊表會撈不到東西，一定要改讀新位置
D. 表不變	跟 R/3 一樣	照舊

Proxy 是什麼：官方名稱是「替代物件 (Replacement Object)」。SAP 在 DDIC 裡幫舊表指定一個 CDS View，程式用 Open SQL **SELECT** 這張舊表時，系統自動把讀取轉到那個 CDS View；舊程式一行不用改，讀到的數字是從新表即時算出來的。這只對**讀取**有效：**INSERT / UPDATE / DELETE** 不會轉，Native SQL、AMDP 也不會轉 (官方文件 **ABENDDIC_REPLACEMENT_OBJECTS**)。對有 Proxy 的表下 **SELECT**，語法檢查會出現警告，提醒你這次讀取被轉走了。

本系統實測：舊表在 **S/4HANA 1909** 的真實狀態 (2026-10-01 查 **DD02L**、**DD25L**、**DDLDEPENDENCY**，並統計筆數)

同樣是「舊表名還查得到」，底層其實分成三種，SE11 打開看起來差不多，差別要看技術設定：

A. 舊表名本身已經變成 CDS View，沒有實體表了 (DD02L-TABCLASS = VIEW)。SAP 用 CDS View 取代舊表，並讓它的 SQL View 名稱沿用舊表名，所以 **SELECT * FROM bsis** 照樣能編譯、執行，資料是從 **ACDOCA** 等新表即時算出來的：

舊表名	原本內容	現在是哪個 CDS View
BSIS 、 BSAS	總帳科目未清 / 已清項索引	BSIS_DDL 、 BSAS_DDL
BSID 、 BSAD	客戶未清 / 已清項索引	BSID_DDL 、 BSAD_DDL
BSIK 、 BSAK	供應商未清 / 已清項索引	BSIK_DDL 、 BSAK_DDL
GLT0	總帳科目合計 (舊總帳)	GLT0_DDL
FAGLFLEXT	總帳合計 (新總帳)	V_FAGLFLEXT_DDL
COSP 、 COSS	CO 成本合計	V_COSP_DDL 、 V_COSS_DDL

B. 舊表還是透明表，但掛了 Proxy (DD02L-TABCLASS = TRANSP，VIEWREF 欄位有值)。**SELECT** 這張表會被轉去讀 Proxy CDS View：

舊表	Proxy CDS View	讀到的資料從哪來
MKPF 、 MSEG	NSDM_DDL_MKPF 、 NSDM_DDL_MSEG	已由 MATDOC 取代：物料憑證只寫進 MATDOC (一列同時帶表頭與明細欄位)，表名留著只為了讓舊程式還能讀。本系統 SELECT COUNT(*) FROM mkpf 是 6,180 筆、 MATDOC 是 12,381 筆：一張憑證在 MATDOC 有多列明細，Proxy 把它們整理回一筆 MKPF 表頭
MARC 、 MARD 、 MCHB	NSDM_DDL_MARC 、 NSDM_DDL_MARD 、 NSDM_DDL_MCHB	主檔欄位讀原表，庫存數量從 MATDOC 即時加總

舊表	Proxy CDS View	讀到的資料從哪來
	NSDM_DDL_MCHB	
MKOL、MSKA、MSLB	NSDM_DDL_MKOL、 NSDM_DDL_MSKA、 NSDM_DDL_MSLB	同上 (寄售、銷售單、轉包特殊庫存)
MBEW	MBV_MBEW	物料評價
FAGLFLEXA	FGL_FAGLFLEXA	ACDOCA
COEP	V_COEP	ACDOCA (統計過帳等資料仍存在 COEP 本身)
ANEP、ANLC	FAA_ANEP、FAA_ANLC	ACDOCA 等新資產會計表

C. 舊表還在、也沒有 Proxy，但已經不寫資料：本系統 **VBUK**、**VBUP**、**KONV** 都是 **0** 筆，讀它們一定撈不到東西，而且不會報錯：

舊表	資料改到哪裡
VBUK 、 VBUP (單據狀態)	狀態欄位搬回各單據：銷售單 → VBAK / VBAP ，交貨單 → LIKP / LIPS ，發票 → VBRK
KONV (單據定價條件)	PRCD_ELEMENTS (本系統 134,115 筆)；也可以讀 SAP 提供的 CDS View V_KONV (欄位排列沿用 KONV)

D. 沒有改變：**BKPF** (憑證表頭)、**BSEG** (原始憑證與未清項管理)、**VBAK** / **VBAP** 等單據表照舊。**KNA1**、**LFA1** 也還是透明表，只是客戶 / 供應商一律透過 Business Partner (交易碼 **BP**) 建立，主資料在 **BUT000** 等表，**KNA1** / **LFA1** 會同步寫入，舊程式照樣讀得到。

自己判斷一張表屬於哪一種：SE11 打開看它是 Table 還是 View；是 Table 就再看有沒有 Proxy (替代物件)；或像本講一樣查 **DD02L** 的 **TABCLASS** 與 **VIEWREF**。最後一定要實際看一下筆數，C 類這種「表在、沒資料、不報錯」最容易誤判。

資料來源：本系統字典表實測；概念說明參考 SAP Help「Universal Journal: FAQ」、「Archiving Material Documents (MM-IM)」、「New Simplified Data Model (NSDM) for Inventory Management Tables」，以及 SAP Community「Changes in Sales & Distribution (SD) in SAP S/4HANA w.r.t ECC」。

寫程式時的原則：

1. 新程式直接讀新表 (**ACDOCA**、**MATDOC**、**PRCD_ELEMENTS**)，不要讀 A、B 類舊表：每次都要即時計算，資料量大時比直接讀新表慢。
2. A、B 類舊表只能讀、不能寫。
3. C 類舊表 (**VBUK**、**VBUP**、**KONV**) 讀了也是空的，舊程式搬到 S/4HANA 時要特別檢查。
4. 照著網路上或舊文件的範例寫程式時，先用 SE11 確認那張表在 S/4HANA 的狀態，不要假設 R/3 的做法還成立。
5. 庫存數量要讀 **MARD-LABST** 這類欄位時，記得它是即時算的，大量讀取要注意效能。

3. SELECT 語法全景

```
SELECT 欄位清單  
FROM 資料表  
INTO 目的地  
[UP TO n ROWS]  
WHERE 條件  
[ORDER BY 排序欄位].
```

3.1 撈多筆進內表 (最常用)

```
DATA gt_carriers TYPE STANDARD TABLE OF scarr.  
  
SELECT * FROM scarr  
      INTO TABLE gt_carriers.  
  
IF sy-subrc <> 0.  
    WRITE / '查無資料'.  
ENDIF.  
WRITE: / '共撈到', sy-dbcnt, '筆'.      " sy-dbcnt : 這次 SELECT 的筆數
```

3.2 指定欄位 (實務建議)

只撈需要的欄位，目的結構的欄位順序與型別要對得上：

```
TYPES: BEGIN OF ty_carr,  
        carrid  TYPE scarr-carrid,  
        carrname TYPE scarr-carrname,  
    END OF ty_carr.  
DATA gt_carr TYPE STANDARD TABLE OF ty_carr.  
  
SELECT carrid carrname FROM scarr  
      INTO TABLE gt_carr.
```

SELECT * 在練習可以，正式程式盡量列欄位：少傳輸、少記憶體，而且表加欄位時不會莫名多撈。欄位對不齊的問題之後用 **INTO CORRESPONDING FIELDS OF** (講義 11) 解。

3.3 SELECT SINGLE : 讀一筆

已知完整鍵、只要一筆時用：

```
DATA gs_carrier TYPE scarr.  
SELECT SINGLE * FROM scarr  
      INTO gs_carrier  
      WHERE carrid = 'AA'.  
IF sy-subrc = 0.
```

```
WRITE: / gs_carrier-carrid, gs_carrier-carrname.  
ENDIF.
```

3.4 WHERE 與 UP TO n ROWS

```
SELECT * FROM sflight  
  INTO TABLE gt_flights  
  UP TO 50 ROWS           " 最多 50 筆 ( 試跑大表的保命符 )  
  WHERE carrid = 'AA'  
        AND fldate >= '20260101'  
        AND seatsocc > 0.
```

WHERE 可用 =、<、>、>=、<、<=、BETWEEN、LIKE (% 萬用字元)、IN (講義 7 搭配 SELECT-OPTIONS)。字元欄位比對區分大小寫，資料庫存大寫就要用大寫比。

3.5 ORDER BY：讓資料庫排好序再回傳

不寫 ORDER BY，撈回來的順序是不確定的：可能剛好照主鍵排，也可能不是，同一句 SELECT 執行兩次順序都可能不同 (官方文件 ABAPORDERBY_CLAUSE)。報表需要固定順序，就要自己指定。

ORDER BY 寫在 WHERE 之後、句點之前：

```
SELECT * FROM sflight  
  INTO TABLE gt_flights  
  WHERE carrid = 'LH'  
  ORDER BY fldate DESCENDING.      " 航班日期由新到舊
```

寫法	意思
ORDER BY fldate	依 fldate 遞增 (由小到大)，不寫方向就是遞增，也可以明寫 ASCENDING
ORDER BY fldate DESCENDING	依 fldate 遞減 (由大到小)
ORDER BY carrid price DESCENDING	多個欄位用空格分隔，先依 carrid 遞增，同一家航空公司再依 price 遞減；方向寫在各欄位後面，只管那一欄
ORDER BY PRIMARY KEY	依這張表的主鍵遞增排序，不用自己列出主鍵欄位

搭配 UP TO n ROWS 取「前幾名」：資料庫先排序，再取前 n 筆，所以下面這句撈到的是 LH 日期最新的 3 班：

```
SELECT * FROM sflight  
  INTO TABLE gt_flights  
  UP TO 3 ROWS  
  WHERE carrid = 'LH'  
  ORDER BY fldate DESCENDING.
```

沒有 `ORDER BY` 的 `UP TO 3 ROWS` 只是「任意 3 筆」，不是最新的 3 筆。

`SELECT SINGLE` 不能加 `ORDER BY` (語法錯誤)。要「符合條件的最新一筆」，用第 4 節的 `SELECT ... ENDSELECT` 加 `ORDER BY`，讀到第一筆就 `EXIT` (講義 13 的傳票清單就用這招找最近一任會計主管)：

```
SELECT * FROM sflight
  INTO gs_flight
  WHERE carrid = 'LH'
        AND fldate <= sy-datum
  ORDER BY fldate DESCENDING.      " 最新的排在最前面
  EXIT.                            " 讀到第一筆就離開，sy-subrc = 0
ENDSELECT.
```

`ORDER BY` 還是講義 5 的 `SORT`：資料撈回來就要照這個順序用，直接 `ORDER BY`；撈回來之後還要依不同欄位重排好幾次，或要排序的欄位是 `ABAP` 算出來的，就撈回內表再 `SORT`。

以上四種寫法 2026-09-30 用驗證程式 `ZR_TR06_ORDERBY_CHK` 實跑確認。多個排序欄位用空格分隔是傳統寫法；講義 26 的新式寫法改用逗號分隔。

4. SELECT ... ENDSELECT (看得懂即可)

舊程式常見的逐筆迴圈式撈取：

```
SELECT * FROM scarr INTO gs_carrier.
  WRITE: / gs_carrier-carrid, gs_carrier-carrname.
ENDSELECT.
```

每一圈跟資料庫要一筆，效能差；維護時看得懂就好，新程式一律 `INTO TABLE` 一次撈回，再用 `LOOP` 處理。同理，絕對不要在 `LOOP` 裡面對每筆再下 `SELECT` (講義 11 用 `JOIN` 解決這個需求)。

5. 常見錯誤與陷阱

症狀	原因
練習系統撈不到航班資料	沒跑過 <code>SAPBC_DATA_GENERATOR</code>
<code>WHERE</code> 比對不到明明存在的值	大小寫不符 (資料庫存大寫) 或欄位有前導零 (<code>n</code> 型別)
指定欄位 <code>SELECT</code> 後資料錯位	目的結構欄位順序/型別跟欄位清單不一致
<code>SELECT SINGLE</code> 撈到「不知道哪一筆」	<code>WHERE</code> 沒給完整鍵——條件不唯一時它任取一筆
程式在大表上跑不完	忘了 <code>WHERE / UP TO n ROWS</code> ，全表掃描
報表順序每次跑都不一樣	沒寫 <code>ORDER BY</code> ，資料庫回傳順序不保證
<code>UP TO 3 ROWS</code> 撈到的不是最新 3 筆	沒加 <code>ORDER BY ... DESCENDING</code> ，取到的是任意 3 筆
自己 <code>WHERE mandt = ...</code>	不需要， <code>Open SQL</code> 自動處理 <code>client</code>

6. 課堂練習

完成 [ex06](#)：確認測試資料、SELECT SCARR 全表與指定欄位、SELECT SINGLE 讀單筆、用 WHERE 與 UP TO n ROWS 撈 SFLIGHT，全程檢查 sy-subrc / sy-dbcnt。